CptCodingTemplates

计算几何

Point

```
class PT{
public:
    db fs , sc;
    PT(db x = 0 , db y = 0) : fs(x) , sc(y) {}

    friend PT operator+(const PT& a , const PT& b){ return PT(a.fs + b.fs , a.sc +
b.sc); }
    friend PT operator-(const PT& a , const PT& b){ return PT(a.fs - b.fs , a.sc -
b.sc); }
    friend PT operator*(const db& t , const PT& a){ return PT(t * a.fs , t *
a.sc); }
    friend PT operator*(const PT& a , const db& t){ return PT(t * a.fs , t *
a.sc); }
    friend db operator*(const PT& a , const PT& b){ return a.fs * b.sc - a.sc *
b.fs; } // cross
    friend db operator%(const PT& a , const PT& b){ return a.fs * b.fs + a.sc *
b.sc; } // dot

    db len()     { return sqrt(fs * fs + sc * sc); }
    db ang(PT& b){ return acos(*this % b / this -> len() / b.len()); }
    db dis(PT& b){ return sqrt(1.0 * (b.fs - fs) * (b.fs - fs) + (b.sc -sc) *
(b.sc - sc)); }
};

PT GetIntsct(PT a , PT DA , PT b , PT DB){ // point ---direction--->
    db ratio = (a - b) * DB / (DB * DA);
    return a + DA * ratio;
}

void PolarSort(vector<PT>& x){
    sort(all(x) , [&](const PT& a , const PT& b){
        db ag1 = atan2(a.sc , a.fs) , ag2 = atan2(b.sc , b.fs);
        if(ag1 < 0)ag1 += 2 * PI;
        if(ag2 < 0)ag2 += 2 * PI;
        return ag1 < ag2;
    });
}

void PolarSort_CrossProductVer(vector<PT>& x){
    auto half = [](const PT& a) -> int {
        return (a.sc < 0 || (a.sc == 0 && a.fs < 0)) ? 1 : 0;
    };
    sort(all(x) , [&](const PT& a, const PT& b){
        if(half(a) != half(b)) return half(a) < half(b);
        db cross = a * b;
```

```
        return cross > EPS;
    });
}
```

Andrew凸包

```
vector<PT> andrew(vector<PT>& ps){
    sort(all(ps) , [](PT& a , PT& b){
        if(a.fs == b.fs)return a.sc < b.sc;
        else return a.fs < b.fs;
    });
    vector<PT>res;
    for(int i=0;i<(int)ps.size();++i){
        while(res.size() >= 2 && (res[res.size() - 1] - res[res.size() - 2]) *
(ps[i] - res[res.size() - 2]) <= 0)res.pop_back();
        res.push_back(ps[i]);
    }
    int sz = res.size();
    for(int i=ps.size() - 2;i>=0;--i){
        while(res.size() >= sz + 1 && (res[res.size() - 1] - res[res.size() - 2])
* (ps[i] - res[res.size() - 2]) <= 0)res.pop_back();
        res.push_back(ps[i]);
    }
    res.pop_back(); // delete the start
    return res;
}
```

半平面交

```
class LN{
public:
    PT p , d;
    db ag;
    LN() {}
    LN(PT p , PT d) : p(p) , d(d) { ag = atan2(d.sc , d.fs); }
    bool operator<(const LN& b) const {
        if(fabs(ag - b.ag) > EPS)return ag < b.ag;
        return d * (b.p - p) > EPS;
    }
};

bool check(LN a , LN b , LN c){
    PT p = GetIntsct(a.p , a.d , b.p , b.d);
    return c.d * (p - c.p) < -EPS;
}

vector<PT> HalfPlaneIntersecion(vector<LN>& L){
    sort(all(L));
```

```
    deque<LN>q;
    for(int i=0;i<(int)L.size();++i){
        if(i < (int)L.size() - 1 && fabs(L[i].ag - L[i + 1].ag) < EPS)continue;
        while(q.size() > 1 && check(q.back() , q[q.size() - 2] ,
L[i]))q.pop_back();
        while(q.size() > 1 && check(q.front() , q[1] , L[i]))q.pop_front();
        q.push_back(L[i]);
    }
    while(q.size() > 1 && check(q.back() , q[q.size() - 2] ,
q.front()))q.pop_back();
    while(q.size() > 1 && check(q.front() , q[1] , q.back()))q.pop_front();
    if(q.size() < 3) return {};
    vector<PT> ans;
    for(int i=0;i<(int)q.size() - 1;++i)ans.push_back(GetIntsct(q[i].p, q[i].d ,
q[i + 1].p , q[i + 1].d));
    ans.push_back(GetIntsct(q.back().p , q.back().d , q.front().p , q.front().d));
    return ans;
}
```

数据结构

树状数组

```
class BIT{
private:
    int n;
    vector<int>tr;
    int lowbit(int x){ return x & -x; }
public:
    BIT(vector<int>a){
        tr.assign(a.size() , 0);
        n = a.size() - 1;
        for(int i=1;i<=n;++i)add(i , a[i]);
    }
    BIT(int sz) : n(sz) , tr(sz + 1 , 0) {}
    void add(int id , int x){
        while(id <= n)tr[id] += x , id += lowbit(id);
    }
    int askpre(int id){
        int res = 0;
        while(id)res += tr[id] , id -= lowbit(id);
        return res;
    }
};

class CompleteBIT{
private:
    int n;
    BIT tr1 , tr2;
    int lowbit(int x){ return x & -x; }
    void add(int id , int x){
```

```
        tr1.add(id , x);
        tr2.add(id , x * id);
    }
public:
    CompleteBIT(int n) : tr1(n) , tr2(n) , n(n){}
    CompleteBIT(vector<int>& a) : n(a.size() - 1) , tr1(n) , tr2(n){ for(int
i=1;i<=n;++i)addrg(i , i , a[i]); }

    void addrg(int l , int r , int x){
        add(l , x) , add(r + 1 , -x);
    }

    int askpre(int id){
        return (id + 1) * tr1.askpre(id) - tr2.askpre(id);
    }

    int askrg(int l , int r){
        return askpre(r) - askpre(l - 1);
    }
};
```

并查集

```
class DSU{
private:
    vector<int>fa , sz;
public:
    DSU(int n){
        fa.assign(n + 1 , 0); sz.assign(n + 1 , 1);
        iota(all(fa) , 0);
    }
    int find(int x){
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }
    bool same(int x , int y){
        return find(x) == find(y);
    }
    void merge(int x , int y){
        x = find(x); y = find(y);
        if(x == y)return;
        if(sz[x] < sz[y])swap(x , y);
        fa[y] = x;
        sz[x] += sz[y];
    }
    int size(int x){
        return sz[find(x)];
    }
};

class DSU_Rollbackable{
private:
```

```
    vector<int>fa , sz;
    vector<pii>st;
    DSU_Rollbackable(int n){
        fa.assign(n + 1 , 0);
        sz.assign(n + 1 , 1);
        iota(all(fa) , 0);
    }
    int find(int x){
        while(fa[x] != x)x = fa[x];
        return x;
    }
    void merge(int x , int y){
        x = find(x); y = find(y);
        if(x == y)return;
        if(sz[x] < sz[y])swap(x , y);
        st.push_back({y , x});
        sz[x] += sz[y];
        fa[y] = x;
    }
    int cur(){
        return st.size();
    }
    void rollback(int k){
        while(st.size() > k){
            int y = st.back().fs;
            int x = st.back().sc;
            st.pop_back();
            fa[y] = y;
            sz[x] -= sz[y];
        }
    }
};
```

对顶堆维护中位数

```
class MaxMinHeap{
private:
    multiset<int>low , high;
    //low : 1 ~ (n + 1) / 2 , high : (n + 1) / 2 + 1 ~ n
    //median : *low.rbegin();
    void balance(){
        while(low.size() > high.size() + 1){
            high.insert(*low.rbegin());
            low.erase(-- low.end());
        }
        while(low.size() < high.size()){
            low.insert(*high.begin());
            high.erase(high.begin());
        }
    }
public:
```

```
    void add(int x){
        if(!high.empty() && x >= *high.begin())high.insert(x);
        else low.insert(x);
        balance();
    }
    void del(int x){
        if(high.count(x))high.erase(high.find(x));
        else if(low.count(x))low.erase(low.find(x));
        balance();
    }
    int get(){
        return *low.rbegin();
    }
};
```

线段树

```
class Tag{
public:
    // Member Variable ...
    Tag() {}
    bool empty() const {
        // How is the Tag empty ...
    }
    void apply(const Tag &t) & {
        // Tag merge ...
    }
};

class Info{
public:
    // Member Variable ...
    Info() {}
    void apply(const Tag &t) & {
        // How Tag apply to the data ...
    }
};

Info operator+(const Info &a, const Info &b){
    Info res;
    // Segment Merge ...
    return res;
}

template<class Info, class Tag>
class LazySegmentTree{
public:
    int n;
    vector<Info> info;
    vector<Tag> tag;
```

```
    LazySegmentTree(int n_) : n(n_) {
        info.assign(4 * n + 1, Info());
        tag.assign(4 * n + 1, Tag());
    }

    template<class T>
    void build(const vector<T>& a, int p, int l, int r) {
        if (l == r) {
            info[p] = Info(a[l]);
            return;
        }
        int m = (l + r) / 2;
        build(a, 2 * p, l, m);
        build(a, 2 * p + 1, m + 1, r);
        pull(p);
    }

    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }

    void push(int p) {
        if (tag[p].empty()) return;
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }

    void modify(int p, int l, int r, int x, const Info &v) {
        if (l == r) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x <= m) modify(2 * p, l, m, x, v);
        else modify(2 * p + 1, m + 1, r, x, v);
        pull(p);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= x && r <= y) return info[p];
        int m = (l + r) / 2;
        push(p);
        if (y <= m) return rangeQuery(2 * p, l, m, x, y);
        if (x > m) return rangeQuery(2 * p + 1, m + 1, r, x, y);
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m + 1, r, x,
y);
    }
```

```
    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x <= m) rangeApply(2 * p, l, m, x, y, v);
        if (y > m) rangeApply(2 * p + 1, m + 1, r, x, y, v);
        pull(p);
    }

    template<class T>
    void build(const vector<T>& a) { build(a, 1, 1, n); }
    void modify(int x, const Info &v) { modify(1, 1, n, x, v); }
    Info rangeQuery(int l, int r) { return rangeQuery(1, 1, n, l, r); }
    void rangeApply(int l, int r, const Tag &v) { rangeApply(1, 1, n, l, r, v); }
};
```

ST表

```
const int inf=1e9+7;
class ST{
private:
    vector<vector<int> >st;
    string tp;
public:
    ST(vector<int>a,string type){//1<=i<=n
        tp=type; assert(tp=="max" || tp=="min");
        int n=a.size()-1;
        st.assign(n+1,vector<int>(20,inf));
        for(int i=1;i<=n;++i)st[i][0]=a[i];
        if(tp=="max"){
            for(int j=1;j<=20;++j){
                for(int i=1;i+(1<<j)-1<=n;++i){
                    st[i][j]=max(st[i][j-1],st[i+(1<<(j-1))][j-1]);
                }
            }
        }else if(tp=="min"){
            for(int j=1;j<=20;++j){
                for(int i=1;i+(1<<j)-1<=n;++i){
                    st[i][j]=min(st[i][j-1],st[i+(1<<(j-1))][j-1]);
                }
            }
        }
    }

    int rg(int l,int r){
        int k=(int)log2(r-l+1);
        if(tp=="max")return max(st[l][k],st[r-(1<<k)+1][k]);
```

```
        else return min(st[l][k],st[r-(1<<k)+1][k]);
    }
};

void solve(){
    int n;cin >> n;
    vector<int>a(n+1);
    for(int i=1;i<=n;++i)cin >> a[i];
    int q;cin >> q;
    ST st1(a,"max"),st2(a,"min");
    while(q--){
        int t,l,r;cin >> t >> l >> r;
        if(t==0)cout << st1.rg(l,r) <<'\n';
        else if(t==1)cout << st2.rg(l,r) << '\n';
    }
}
```

图论

倍增最近公共祖先

```
int n , m;
vector<int> dep;
vector<vector<int> > fa , g;

void dfs(int now , int from){
    dep[now] = dep[from] + 1;
    fa[now][0] = from;
    for(int i=1;i<=25;++i)fa[now][i] = fa[fa[now][i - 1]][i - 1];
    for(auto nx : g[now])if(nx != from)dfs(nx , now);
}

int lca(int u , int  v){
    if(dep[u] < dep[v])swap(u , v);
    for(int i=25;i>=0;--i)if(dep[fa[u][i]] >= dep[v])u = fa[u][i];
    if(u == v)return v;
    for(int i=25;i>=0;--i)if(fa[u][i] != fa[v][i])u = fa[u][i] , v = fa[v][i];
    return fa[v][0];
}

void solve(){
    cin >> n >> m;
    dep.assign(n + 1 , 0); fa.assign(n + 1 , vector<int>(26 , 0)); g.assign(n + 1
, vector<int>());
    for(int i=1;i<=m;++i){
        int u,v;cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    dfs(1 , 0);
    int q;cin >> q;
```

```
    while(q --){
        int u,v;cin >> u >> v;
        cout << lca(u , v) << '\n';
    }
}
```

Tarjan最近公共祖先

```
int n , q;
vector<vector<int> >g;
vector<vector<pii> >qry;
vector<int>fa , ans;
vector<bool>vis;

int find(int x){
    if(x == fa[x])return x;
    else return fa[x] = find(fa[x]);
}

void dfs(int u){
    vis[u] = true;
    for(auto v:g[u]){
        if(!vis[v]){
            dfs(v);
            fa[v] = u;
        }
    }
    for(auto i:qry[u]){
        int v = i.fs , id = i.sc;
        if(vis[v])ans[id] = find(v);
    }
}

signed main(){
    cin >> n >> q;
    g.assign(n + 1 , vector<int>());
    qry.assign(n + 1  , vector<pii>());
    for(int i=1;i<=n - 1;++i){
        int u , v;cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    for(int i=1;i<=q;++i){
        int u , v;cin >> u >> v;
        qry[u].push_back({v , i});
        qry[v].push_back({u , i});
    }
    ans.assign(q + 1 , 0);
    fa.assign(n + 1  , 0); iota(all(fa) , 0);
    vis.assign(n + 1 , false);
    dfs(1);
```

```
    for(int i=1;i<(int)ans.size();++i)cout << ans[i] << '\n';
    return 0;
}
```

Kahn拓扑排序

```
vector<vector<int> > g;
vector<int>res , cond;
bool cyc = false;

void dfs(int now){
    if(cyc)return;
    cond[now] = 1;
    for(auto i:g[now]){
        if(cond[i] == 0)dfs(i);
        else if(cond[i] == 1){
            cyc = true;
            return;
        }
    }
    cond[now] = 2;
    res.push_back(now);
}

signed main(){
    int n,m;cin >> n >> m;
    g.assign(n + 1 , vector<int>());
    cond.assign(n + 1 , 0);
    res.clear();
    for(int i=1;i<=m;++i){
        int u,v;cin >> u >> v;
        g[u].push_back(v);
    }
    for(int i=1;i<=n;++i)if(cond[i] == 0)dfs(i);
    reverse(all(res));
    if(!cyc){
        for(auto i:res)cout << i << ' '; cout << '\n';
    }else NO
    return 0;
}
```

DFS拓扑排序

```
vector<vector<int> > g;
vector<int>res , cond;
bool cyc = false;

void dfs(int now){
```

```
    if(cyc)return;
    cond[now] = 1;
    for(auto i:g[now]){
        if(cond[i] == 0)dfs(i);
        else if(cond[i] == 1){
            cyc = true;
            return;
        }
    }
    cond[now] = 2;
    res.push_back(now);
}

signed main(){
    int n,m;cin >> n >> m;
    g.assign(n + 1 , vector<int>());
    cond.assign(n + 1 , 0);
    res.clear();
    for(int i=1;i<=m;++i){
        int u,v;cin >> u >> v;
        g[u].push_back(v);
    }
    for(int i=1;i<=n;++i)if(cond[i] == 0)dfs(i);
    reverse(all(res));
    if(!cyc){
        for(auto i:res)cout << i << ' '; cout << '\n';
    }else NO
    return 0;
}
```

BellmanFord

```
struct edge{
    int from,to,cost;
};

const int MAXE=50010;
const int MAXV=50010;
const int INF=1e9+7;

edge es[MAXE];
int d[MAXV];
int V,E;

void BellmanFord(int s){
    fill(d,d+V,INF);
    d[s]=0;
    while(1){
        bool update=false;
        for(int i=0;i<E;++i){
            edge e=es[i];
```

```
            if(d[e.from]!=INF && d[e.to]>d[e.from]+e.cost){
                d[e.to]=d[e.from]+e.cost;
                update=true;
            }
        }
        if(!update)break;// 直到遍历一次没有更新：找到最优解
    }
}

bool FindNegativeLoop(){
    memset(d,0,sizeof(d));
    for(int i=0;i<V;++i){
        for(int j=0;j<E;++j){
            edge e=es[j];
            if(d[e.to]>d[e.from]+e.cost){
                d[e.to]=d[e.from]+e.cost;
                if(i==V-1)return true;// 如果第V次仍然更新，则存在负圈
            }
        }
    }
    return false;
}

void solve(){
    cin>>V>>E;
    for(int i=0;i<E;++i){
        int u,v,cost;cin>>u>>v>>cost;
        es[i].cost=cost;es[i].from=u;es[i].to=v;
    }
    int s;cin>>s;//起点
    BellmanFord(s);
    if(FindNegativeLoop())cout<<"ExistNegativeLoop"<<'\n';
    else cout<<"NoNegativeLoop"<<'\n';
}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    solve();
    return 0;
}
```

Dijkstra次短路

```
typedef pair<int,int>P;
const int INF=1e9+7;
int dijkstra(vector<P>g[],int s,int t,int V){
    priority_queue<P>q;
    q.push({0,s});
    vector<int>dis(V,INF);
    while(!q.empty()){
```

```
        P now=q.top();q.pop();

    }
    return dis[t];
}

void solve(){
    int V,E;cin>>V>>E;
    int s,t;cin>>s>>t;
    vector<P>g[V];
    for(int i=0;i<V;++i){
        int u,v,c;cin>>u>>v>>c;
        g[u].push_back({c,v});
        g[v].push_back({c,u});
    }
    dijkstra(g,s,t,V);

}
```

Dijkstra路径还原

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N=70000;
const int INF=1e9+7;
const int NIL=-1000000007;
const double pi=acos(-1.0);

//Vertex indexes start with 1

typedef pair<int, int>P;//1st:cost , 2nd:to

int V, E, s, t;
vector<P>edge[N];
int pre[N];
int dis[N];

void dijkstra(int s) {
    priority_queue<P, vector<P>, greater<P> >que;
    fill(dis, dis + V + 1, INF); dis[s] = 0;
    fill(pre,pre+V,-1);
    que.push(P(0, s));
    while (!que.empty()) {
        P p = que.top(); que.pop();
        int v = p.second;
        if (dis[v] < p.first)continue;
        for (int i = 0; i < (int)edge[v].size(); ++i) {
            P e = edge[v][i];
            if (dis[e.second] > dis[v] + e.first) {
                dis[e.second] = dis[v] + e.first;
```

```
                pre[e.second]=v;
                que.push(P(dis[e.second], e.second));
            }
        }
    }
}

vector<int> GetPath(int t){
    vector<int>path;
    for(;t!=-1;t=pre[t])path.push_back(t);//直到走到s(起点)
    reverse(path.begin(),path.end());
    return path;
}

signed main() {
    cin >> V >> E >> s >> t;
    for (int i = 0; i < E; ++i) {
        int u, v, w; cin >> u >> v >> w;
        edge[u].push_back(P(w, v)); edge[v].push_back(P(w, u));
    }
    dijkstra(s);
    cout<<dis[t]<<'\n';
    vector<int>path=GetPath(t);
    for(auto i:path)cout<<i<<' ';
    cout<<'\n';
    return 0;
}
```

Kruskal

```
const int N=500010;
const int MAXE=500010;

struct edge{
    int u,v,cost;
};

int fa[N];
int rk[N];

void init(int n){
    for(int i=0;i<n;++i){
        fa[i]=i;
        rk[i]=0;
    }
}

int find(int x){
    if(fa[x]==x){
        return x;
    }else{
```

```
        return fa[x]=find(fa[x]);
    }
}

void merge(int x,int y){
    x=find(x);
    y=find(y);
    if(x==y)return;
    if(rk[x]<rk[y]){
        fa[x]=y;
    }else {
        fa[y]=x;
        if(rk[x]==rk[y])rk[x]++;
    }
}

bool same(int x,int y){
    return find(x)==find(y);
}

bool cmp(edge e1,edge e2){
    return e1.cost<e2.cost;
}

edge es[MAXE];
int V,E;

int kruskal(){
    sort(es,es+E,cmp);
    init(V);
    int res=0;
    for(int i=0;i<E;++i){
        edge e=es[i];
        if(!same(e.u,e.v)){
            merge(e.u,e.v);
            res+=e.cost;
        }
    }
    return res;
}

void solve(){
    cin>>V>>E;
    for(int i=0;i<E;++i){
        cin>>es[i].u>>es[i].v>>es[i].cost;
    }
    cout<<kruskal()<<'\n';
}
```

字符串

Trie

```
class Trie{
public:
    vector<arr<int, 26> >ch;
    vector<int> cnt , pass;
    Trie(){
        ch.push_back({});
        ch[0].fill(0);
        cnt.push_back(0);
        pass.push_back(0);
    }

    void insert(const string& s){
        int u = 0;
        pass[u]++;
        for (char c:s){
            int x = c - 'a';
            if(!ch[u][x]){
                ch.push_back({});
                ch.back().fill(0);
                cnt.push_back(0);
                pass.push_back(0);
                ch[u][x] = ch.size() - 1;
            }
            u = ch[u][x];
            pass[u]++;
        }
        cnt[u]++;
    }

    int search(const string& s){
        int u = 0;
        for(char c:s){
            int x = c - 'a';
            if(!ch[u][x]) return 0;
            u = ch[u][x];
        }
        return cnt[u];
    }

    int askpre(const string& s){
        int u = 0;
        for(char c:s){
            int x = c - 'a';
            if(!ch[u][x])return 0;
            u = ch[u][x];
        }
        return pass[u];
    }

    void erase(const string& s){
        if(!search(s))return;
        int u = 0;
```

```
        pass[u]--;
        for(char c:s){
            int x = c - 'a';
            u = ch[u][x];
            pass[u]--;
        }
        cnt[u]--;
    }
};

void solve(){
    int n;cin >> n;
    Trie trie;
    while(n--){
        int op; string s;
        cin >> op >> s;
        if(op == 1)trie.insert(s);
        else if(op == 2)trie.erase(s);
        else if(op == 3)cout << (trie.search(s) ? "YES" : "NO") << '\n';
        else cout << trie.askpre(s) << '\n';
    }
}

class TrieNode{
public:
    unordered_map<char , TrieNode*>child;
    // map<char,TrieNode*>child;
    int cnt , cntpre;
    bool isend;
    TrieNode(){cnt = 0; cntpre = 0; isend = false;}
};

class Generic_Trie{
private:
    TrieNode* rt;
public:
    Generic_Trie(){
        rt = new TrieNode();
    }
    void insert(string s){
        TrieNode* now = rt;
        for(auto i:s){
            now -> cntpre++;
            if(now -> child.find(i) == now -> child.end()){
                now -> child[i] = new TrieNode();
            }
            now = now -> child[i];
        }
        now -> cnt++; now -> cntpre++; now -> isend = true;
    }
    int askpre(string s){
        TrieNode* now = rt;
        for(auto i:s){
            if(now -> child.find(i) == now -> child.end())return 0;
```

```
            now = now -> child[i];
        }
        return now -> cntpre;
    }
    int find(string s){
        TrieNode* now = rt;
        for(auto i:s){
            if(now -> child.find(i) == now -> child.end())return false;
            now = now -> child[i];
        }
        return now -> isend;
    }
};

void solve2(){
    Generic_Trie trie;
    int n;cin >> n;
    for(int i=0;i<n;++i){
        string s;cin >> s;
        trie.insert(s);
    }
    int q;cin >> q;
    while(q --){
        string x;cin >> x;
        if(trie.find(x))cout << "Exist" << '\n';
        else cout << "Nope" << '\n';
    }
    cin >> q;
    while(q--){
        string x;cin >> x;
        cout << trie.askpre(x) << '\n';
    }
}
```

KMP

```
class KMP{
private:
    string text;
    vector<int> getnx(string p){
        int n=p.size();
        p=' '+p; vector<int>nx(p.size());
        nx[1]=0;
        for(int i=2,j=0;i<=n;++i){
            while(j && p[i]!=p[j+1])j=nx[j];
            if(p[i]==p[j+1])j++;
            nx[i]=j;
        }
        return nx;
    }
public:
```

```
    KMP(string s){
        text=s; text=' '+text;
    }
    vector<int> match(string p){
        int n=p.size(),m=text.size();
        vector<int>nx=getnx(p);
        p=' '+p;
        vector<int>pos;
        for(int i=1,j=0;i<=m;++i){
            while(j && text[i]!=p[j+1])j=nx[j];
            if(text[i]==p[j+1])j++;
            if(j==n)pos.push_back(i-n+1);
        }
        return pos;
    }
};
```

字符串哈希

```
using ull = unsigned long long;

class StringHash{
private:
    const ll P = 13331;
    vector<ull>poly , hash_val;
public:
    StringHash(string s){
        int n = s.size();
        s = ' ' + s;
        poly.assign(n + 1 , 1); hash_val.assign(n + 1 , 0);
        for(int i=1;i<=n;++i){
            poly[i] = poly[i - 1] * P;
            hash_val[i] = hash_val[i - 1] * P + s[i];
        }
    }
    ull cal(int l , int r){
        return hash_val[r] - hash_val[l - 1] * poly[r - l + 1];
    }
    bool same(int l1 , int r1 , int l2 , int r2){
        return cal(l1 , r1) == cal(l2 , r2);
    }
};
```

数学

矩阵/矩阵快速幂

```
class Mat{
public:
    int n , m , P;
    vector<vector<int> > pos;
    Mat(const vector<vector<int> > & mat , const int & p = MOD) : n(mat.size()) ,
m(mat.empty() ? 0 : mat[0].size()) , pos(mat) , P(p) {}
    Mat(const int & n , const int & m , const int & p = MOD) : n(n) , m(m) , P(p)
{ pos.assign(n , vector<int>(m , 0)); }

    friend Mat operator*(const Mat & A , const Mat & B){
        assert(A.m == B.n && A.P == B.P);
        Mat res(A.n , B.m , A.P);
        for(int i=0;i<A.n;++i){
            for(int j=0;j<B.m;++j){
                for(int k=0;k<A.m;++k){
                    res.pos[i][j] = (res.pos[i][j] + A.pos[i][k] * B.pos[k][j] %
A.P) % A.P;
                }
            }
        }
        return res;
    }
    friend Mat operator+(const Mat & A , const Mat & B){
        assert(A.n == B.n && A.m == B.m && A.P == B.P);
        Mat res(A.n, A.m, A.P);
        for (int i=0;i<A.n;++i)for(int j=0;j<A.m;++j)res.pos[i][j] = (A.pos[i][j]
+ B.pos[i][j]) % A.P;
        return res;
    }
    friend Mat operator*(const Mat & A , const int & x){
        Mat res(A.n , A.m , A.P);
        for(int i=0;i<A.n;++i)for(int j=0;j<A.m;++j)res.pos[i][j] = A.pos[i][j] *
x % A.P;
        return res;
    }
    friend Mat operator*(const int & x , const Mat & A){ return A * x; }

    void set_identity(){ assert(n == m); for(int i=0;i<n;++i)pos[i][i] = 1; }
    void set_zeros()   { for(int i=0;i<n;++i)for(int j=0;j<m;++j)pos[i][j] = 0; }
    void mfpow(int x){
        assert(n == m);
        Mat A = *this , res(n , n , P); res.set_identity();
        while(x){
            if(x & 1)res = res * A;
            A = A * A;
            x >>= 1;
        }
        pos = res.pos;
    }
};
```

阶乘，排列，组合及其逆元

```
class Comb{
private:
    int P;
public:
    vector<int> fact , invf;
    Comb(int n , int p = MOD) : P(p) {
        fact.assign(n + 1 , 1); invf.assign(n + 1 , 1);
        for(int i=1;i<=n;++i)fact[i] = fact[i - 1] * i % P;
        invf[n] = inv(fact[n] , P);
        for(int i=n - 1;i>=1;--i)invf[i] = invf[i + 1] * (i + 1) % P;
    }
    int C(int M , int N){ return fact[M] % P * invf[N] % P * invf[M - N] % P; }
    int A(int M , int N){ return fact[M] % P * invf[M - N] % P; }
    int Q(int M , int N){ return fact[M] % P * inv(N , P) % P * invf[M - N] % P;
};
    int invC(int M , int N){ return inv(C(M , N) , P) % P; }
    int invA(int M , int N){ return inv(A(M , N) , P) % P; }
    int invQ(int M , int N){ return inv(Q(M , N) , P) % P; }
};

class Comb2{
private:
    int P;
public:
    vector<vector<int> > C;
    Comb2(int n , int m , int p = MOD) : P(p) { // long long overflow when n > 66
without P
        C.assign(n + 1 , vector<int>(m + 1 , 0));
        for(int i=0;i<=n;++i)C[i][0] = 1;
        for(int i=1;i<=n;++i)for(int j=1;j<=min(i , m);++j)C[i][j] = (C[i - 1][j -
1] + C[i - 1][j]) % P;
    }
};
```

卡特兰数

$$H_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n} = \frac{4n-2}{n+1}H_{n-1}$$

```
// long long overflow when n > 18 without MOD
```

错排数

```
class Derangement{
private:
```

```
    int P;
public:
    vector<int>D;
    Derangement(int n , int p = MOD) : P(p) {
        D.assign(n + 1 , 0);
        D[0] = D[1] = 0; D[2] = 1;
        for(int i=3;i<=n;++i)D[i] = (i  - 1) * (D[i - 1] + D[i - 2]) % P;
    }
};
```

斯特林轮换数/子集数

```
class StirlingCycle{
private:
    int P;
public:
    vector<vector<int> > S;
    StirlingCycle(int n , int m , int p = MOD) : P(p) { // long long overflow when
n > 20 without MOD
        S.assign(n + 1 , vector<int>(m + 1 , 0));
        for(int i=0;i<=n;++i)S[i][0] = (i == 0);
        for(int i=1;i<=n;++i)for(int j=1;j<=min(i , m);++j)S[i][j] = (S[i - 1][j -
1] % P + (i - 1) * S[i - 1][j] % P) % P;
    }
};


class StirlingSubset{
private:
    int P;
public:
    vector<vector<int> > S;
    StirlingSubset(int n , int m , int p = MOD) : P(p) { // long long overflow
when n > 25 without MOD
        S.assign(n + 1 , vector<int>(m + 1 , 0));
        for(int i=0;i<=n;++i)S[i][0] = (i == 0);
        for(int i=1;i<=n;++i)for(int j=1;j<=min(m , i);++j)S[i][j] = (S[i - 1][j -
1] % P + S[i - 1][j] * j % P) % P;
    }
};
```

斯特林反演

普通幂 $\Longleftrightarrow$ 下降幂

$$x^n=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}x^{\underline{k}}\iff x^{\underline{n}}=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}x^k$$

$$f_n=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}g_k \iff g_n=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}f_k$$

普通幂 $\iff$ 上升幂

$$x^n=\sum_{k=0}^{n}(-1)^{n-k}\begin{Bmatrix}n\\k\end{Bmatrix}x^{\overline{k}} \iff x^{\overline{n}}=\sum_{k=0}^{n}\begin{bmatrix}n\\k\end{bmatrix}x^k$$

$$f_n=\sum_{k=0}^{n}(-1)^{n-k}\begin{Bmatrix}n\\k\end{Bmatrix}g_k \iff g_n=\sum_{k=0}^{n}\begin{bmatrix}n\\k\end{bmatrix}f_k$$

十二重计数法

小球 (n 个)	盒子 (k 个)	任意放	每个盒子至少 1 个 (非空)	每个盒子至多 1 个
不同	不同	k^n	$k!\cdot\begin{Bmatrix}n\\k\end{Bmatrix}$	$n^{\underline{k}}$
相同	不同	$\binom{n+k-1}{k-1}$	隔板法：$\binom{n-1}{k-1}$	$\binom{k}{n}$
不同	相同	$\sum_{i=1}^{k}\begin{Bmatrix}n\\i\end{Bmatrix}$	第二类斯特林数：$\begin{Bmatrix}n\\k\end{Bmatrix}$	$[n\le k]$
相同	相同	$p(n,\le k)$	整数拆分：$p(n-k,\le k)$	$[n\le k]$

快速傅里叶变换

```
using db = double;
const db PI = acos(-1);

void butterfly(vector<complex<db> >& A , int n){
    vector<int>dp(n, 0);
    for(int i=0;i<n;++i)dp[i] = dp[i / 2] / 2 + ((i & 1) ? n / 2 : 0);
    for(int i=0;i<n;++i)if(i < dp[i])swap(A[i] , A[dp[i]]);
}

void FFT(vector<complex<db> >& A , int n){
    butterfly(A , n);
    for(int m=2;m<=n;m <<= 1){
        complex<db> spin{cos(2 * PI / m) , sin(2 * PI / m)};
        for(int i=0;i<n;i += m){
            complex<db> wk = {1 , 0};
            for(int j=0;j<m / 2;++j){
                complex<db> x = A[i + j] , y = A[i + j + m / 2] * wk;
                A[i + j] = x + y;
                A[i + j + m / 2] = x - y;
                wk *= spin;
            }
        }
    }
}
```

```
void IFFT(vector<complex<db> >& A , int n){
    butterfly(A , n);
    for(int m=2;m<=n;m <<= 1){
        complex<db> spin{cos(2 * PI / m) , -sin(2 * PI / m)};
        for(int i=0;i<n;i += m){
            complex<db> wk = {1 , 0};
            for(int j=0;j<m / 2;++j){
                complex<db> x = A[i + j] , y = A[i + j + m / 2] * wk;
                A[i + j] = x + y;
                A[i + j + m / 2] = x - y;
                wk *= spin;
            }
        }
    }
    for(int i=0;i<n;++i)A[i] /= n;
}
```

快速数论变换

```
const int MOD = 998244353;
const int G = 3;

int fpowMOD(int a , int n , int p){
    int res = 1; a %= p;
    while(n){
        if(n & 1)res = res * a % p;
        a = a * a % p;
        n >>= 1;
    }
    return res;
}

void NTT(vector<int>& A , int n , bool inv){
    for(int i = 1 , j = 0;i<n;++i){
        int bit = n >> 1;
        for(;j & bit;bit >>= 1)j ^= bit;
        j ^= bit;
        if(i < j)swap(A[i], A[j]);
    }
    for(int m = 2;m <= n;m <<= 1){
        int wn = inv ? fpowMOD(G , MOD - 1 - (MOD - 1) / m , MOD)
                     : fpowMOD(G , (MOD - 1) / m , MOD);
        for(int i=0;i<n;i += m){
            int wk = 1;
            for (int j=0;j<m / 2;++j){
                int x = A[i + j] , y = A[i + j + m / 2] * wk % MOD;
                A[i + j]         = (x + y) % MOD;
                A[i + j + m / 2] = (x - y + MOD) % MOD;
                wk = wk * wn % MOD;
            }
        }
```

```
    }
    if(inv){
        int ninv = fpowMOD(n, MOD - 2, MOD);
        for (auto& x : A) x = x * ninv % MOD;
    }
}

vector<int> PolyMul(vector<int>& a, vector<int>& b){
    int n = 1;
    int sz = a.size() + b.size() - 1;
    while(n<sz)n <<= 1;

    vector<int>A(n) , B(n);
    for (int i=0;i<(int)a.size();++i)A[i] = a[i];
    for (int i=0;i<(int)b.size();++i)B[i] = b[i];

    NTT(A , n , false);
    NTT(B , n , false);

    for(int i=0;i<n;++i)A[i] = A[i] * B[i] % MOD;

    NTT(A , n , true);

    return vector<int>(A.begin(), A.begin() + sz);
}
```

线性筛

```
class Sieve{
public:
    vector<bool>vs;
    vector<int>pm;
    Sieve(int n){
        vs.assign(n + 1 , false);
        for(int i=2;i<=n;++i){
            if(!vs[i])pm.push_back(i);
            for(int j=0;j<(int)pm.size() && pm[j] <= n / i;++j){
                vs[pm[j] * i] = true;
                if(i % pm[j] == 0)break;
            }
        }
    }
    bool is_prime(int x){
        return x > 2 && !vs[x];
    }
};
```

乘法逆元/快速幂

```
int fpow(int a , int n){
    int res = 1;
    while(n){
        if(n & 1)res *= a;
        a *= a;
        n >>= 1;
    }
    return res;
}

int fpowMOD(int a , int n , int p = MOD){
    int res = 1; a %= p;
    while(n){
        if(n & 1)res = res * a % p;
        a = a * a % p;
        n >>= 1;
    }
    return res;
}

int inv(int x , int p = MOD){ return fpowMOD(x , p - 2 , p); }
```

杂项

i128

```
using i128 = __int128_t;

istream &operator>>(istream &is , i128 &n){
    n = 0;
    string s;
    is >> s;
    for(auto c:s)n = 10 * n + c - '0';
    return is;
}

ostream &operator<<(ostream &os , i128 &n){
    if(n == 0)return os << 0;
    string s;
    while(n > 0){
        s += '0' + n % 10;
        n /= 10;
    }
    reverse(all(s));
    return os << s;
}
```

高精度

```
const int MOD = 998244353;
const int G = 3;
const int Gi = 332748118;

int qpow(int a, int b) {
    int res = 1; a %= MOD;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void ntt(vector<int>& a, int n, int type) {
    static vector<int> r;
    if (r.size() != n) {
        r.resize(n);
        for (int i = 0; i < n; i++) r[i] = (r[i >> 1] >> 1) | ((i & 1) ? (n >> 1)
: 0);
    }
    for (int i = 0; i < n; i++) if (i < r[i]) swap(a[i], a[r[i]]);
    for (int mid = 1; mid < n; mid <<= 1) {
        int wn = qpow(type == 1 ? G : Gi, (MOD - 1) / (mid << 1));
        for (int j = 0; j < n; j += (mid << 1)) {
            int w = 1;
            for (int k = 0; k < mid; k++, w = w * wn % MOD) {
                int x = a[j + k], y = w * a[j + mid + k] % MOD;
                a[j + k] = (x + y) % MOD;
                a[j + mid + k] = (x - y + MOD) % MOD;
            }
        }
    }
    if (type == -1) {
        int inv = qpow(n, MOD - 2);
        for (int i = 0; i < n; i++) a[i] = a[i] * inv % MOD;
    }
}

class BigInt {
private:
    static const int BASE = 100000000;
    static const int WIDTH = 8;
    vector<int> a;
    bool sign;

    void trim() {
        while (a.size() > 1 && a.back() == 0) a.pop_back();
        if (a.empty()) a.push_back(0);
        if (a.size() == 1 && a[0] == 0) sign = false;
    }
```

```
    bool abs_less(const BigInt& b) const {
        if (a.size() != b.a.size()) return a.size() < b.a.size();
        for (int i = a.size() - 1; i >= 0; i--)
            if (a[i] != b.a[i]) return a[i] < b.a[i];
        return false;
    }

    BigInt abs_add(const BigInt& b) const {
        BigInt res; res.a.clear();
        int c = 0;
        for (int i = 0; i < max(a.size(), b.a.size()) || c; i++) {
            if (i < a.size()) c += a[i];
            if (i < b.a.size()) c += b.a[i];
            res.a.push_back(c % BASE);
            c /= BASE;
        }
        res.trim();
        return res;
    }

    BigInt abs_sub(const BigInt& b) const {
        BigInt res = *this;
        for (int i = 0, brw = 0; i < res.a.size(); i++) {
            res.a[i] -= (i < b.a.size() ? b.a[i] : 0) + brw;
            brw = (res.a[i] < 0);
            if (brw) res.a[i] += BASE;
        }
        res.trim();
        return res;
    }

    BigInt abs_mod_div(const BigInt& b, BigInt& r) const {
        if (b.a.size() == 1 && b.a[0] == 0) { r = *this; return BigInt(0); }
        BigInt q(0); r = *this;
        if (abs_less(b)) { r.sign = false; return q; }
        q.a.resize(a.size() - b.a.size() + 1);
        for (int i = a.size() - b.a.size(); i >= 0; i--) {
            BigInt sfd = b;
            sfd.a.insert(sfd.a.begin(), i, 0);
            int low = 0, high = BASE - 1, ansq = 0;
            while (low <= high) {
                int mid = low + (high - low) / 2;
                if (!r.abs_less(sfd * BigInt(mid))) { ansq = mid; low = mid + 1; }
                else high = mid - 1;
            }
            q.a[i] = ansq;
            r = r.abs_sub(sfd * BigInt(ansq));
        }
        q.trim(); r.trim(); r.sign = false;
        return q;
    }

public:
    BigInt(int x = 0) : sign(x < 0) {
```

```
        x = abs(x);
        if (x == 0) a.push_back(0);
        while (x > 0) { a.push_back(x % BASE); x /= BASE; }
        trim();
    }
    BigInt(const string& s) : sign(false) {
        int st = (s[0] == '-' ? 1 : 0);
        if (s[0] == '-') sign = true;
        for (int i = s.length(); i > st; i -= WIDTH) {
            if (i - WIDTH < st) a.push_back(stoll(s.substr(st, i - st)));
            else a.push_back(stoll(s.substr(i - WIDTH, WIDTH)));
        }
        trim();
    }

    bool operator<(const BigInt& b) const {
        if (sign != b.sign) return sign;
        return sign ? b.abs_less(*this) : abs_less(b);
    }
    bool operator>(const BigInt& b) const { return b < *this; }
    bool operator<=(const BigInt& b) const { return !(*this > b); }
    bool operator>=(const BigInt& b) const { return !(*this < b); }
    bool operator==(const BigInt& b) const { return sign == b.sign && a == b.a; }
    bool operator!=(const BigInt& b) const { return !(*this == b); }

    BigInt operator+(const BigInt& b) const {
        if (sign == b.sign) { BigInt res = abs_add(b); res.sign = sign; return
res; }
        if (abs_less(b)) { BigInt res = b.abs_sub(*this); res.sign = b.sign;
return res; }
        BigInt res = abs_sub(b); res.sign = sign; return res;
    }
    BigInt operator-(const BigInt& b) const {
        BigInt t = b; t.sign = !b.sign; return *this + t;
    }

    BigInt operator*(const BigInt& b) const {
        if ((a.size() == 1 && a[0] == 0) || (b.a.size() == 1 && b.a[0] == 0))
return BigInt(0);
        if (a.size() + b.a.size() < 64) {
            BigInt res; res.a.assign(a.size() + b.a.size(), 0);
            for (int i = 0; i < a.size(); i++) {
                int c = 0;
                for (int j = 0; j < b.a.size() || c; j++) {
                    int cur = res.a[i + j] + c + (j < b.a.size() ? a[i] * b.a[j] :
0);
                    res.a[i + j] = cur % BASE; c = cur / BASE;
                }
            }
            res.sign = sign != b.sign; res.trim(); return res;
        }
        vector<int> va, vb;
        for (int x : a) { for (int i = 0; i < WIDTH; i++) { va.push_back(x % 10);
x /= 10; } }
```

```
        for (int x : b.a) { for (int i = 0; i < WIDTH; i++) { vb.push_back(x %
10); x /= 10; } }
        int n = 1, m = va.size() + vb.size() - 1;
        while (n <= m) n <<= 1;
        va.resize(n); vb.resize(n);
        ntt(va, n, 1); ntt(vb, n, 1);
        for (int i = 0; i < n; i++) va[i] = va[i] * vb[i] % MOD;
        ntt(va, n, -1);
        BigInt res; res.a.clear();
        int c = 0;
        for (int i = 0; i < m || c; i++) {
            if (i < m) c += va[i];
            res.a.push_back(c % 10); c /= 10;
        }
        string s = ""; if (sign != b.sign) s += '-';
        for (int i = res.a.size() - 1; i >= 0; i--) s += (char)(res.a[i] + '0');
        return BigInt(s);
    }

    BigInt operator/(const BigInt& b) const {
        BigInt r; BigInt q = abs_mod_div(b, r);
        q.sign = (q.a.size() > 1 || q.a[0] > 0) && (sign != b.sign);
        return q;
    }
    BigInt operator%(const BigInt& b) const {
        BigInt r; abs_mod_div(b, r);
        r.sign = (r.a.size() > 1 || r.a[0] > 0) && sign;
        return r;
    }

    friend istream& operator>>(istream& is, BigInt& n) {
        string s; if (!(is >> s)) return is;
        n = BigInt(s); return is;
    }
    friend ostream& operator<<(ostream& os, const BigInt& n) {
        if (n.sign) os << '-';
        os << n.a.back();
        for (int i = n.a.size() - 2; i >= 0; i--) os << setfill('0') <<
setw(WIDTH) << n.a[i];
        return os;
    }
};

signed main() {
    BigInt a,b;cin >> a >> b;
    cout << a * b << '\n';
    return 0;
}
```

CMakelist

```
cmake_minimum_required(VERSION 3.25)
project(ICPC)

set(CMAKE_C_STANDARD 14)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

file(GLOB_RECURSE SOURCES "*.cpp")

foreach(SRC ${SOURCES})
    get_filename_component(FILENAME ${SRC} NAME_WE)
    add_executable(${FILENAME} ${SRC})
endforeach()
```

循环多线程优化

```
#pragma omp parallel for schedule(dynamic)
for(int i=0;i<n;++i) // A loop with no self-dependency
    // .......
```

```
g++ -O3 -march=native -fopenmp main.cpp -o main
```

编译脚本

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Usage: comp <source.cpp>"
    exit 1
fi

SRC="$1"

if [ ! -f "$SRC" ]; then
    echo "Error: File '$SRC' not found."
    exit 1
fi


BASENAME=$(basename "$SRC" .cpp)
OUTDIR=$(dirname "$SRC")
OUT="$OUTDIR/$BASENAME"

echo "Compiling $SRC -> $OUT ..."
```

```
g++ -O2 -o "$OUT" "$SRC" \
    -std=c++17 \
    -Wall \
    -Wextra \
    -DLOCAL

if [ $? -ne 0 ]; then
    echo "Compilation failed."
    exit 1
fi

echo "OK. Running $OUT ..."
echo "----------------------------------------"

"$OUT"

# echo 'alias comp="~/Desktop/icpc/comp"' >> ~/.bashrc && source ~/.bashrc && echo
"OK"
```